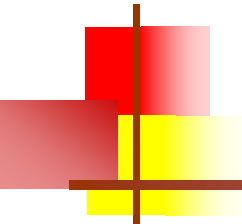


JavaScript

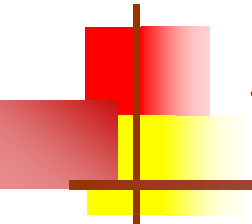
Language Fundamentals





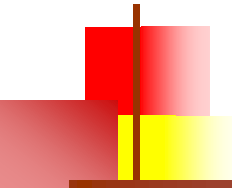
About JavaScript

- **JavaScript** is a **high-level, dynamic scripting language** mainly used to make **web pages interactive**.
- It was developed by **Netscape in 1995**.
- Client-side JavaScript runs primarily in **web browsers**, but it can also run on servers using **Node.js**.
- It follows a **multi-paradigm approach**: procedural, object-oriented, and functional programming.
- JavaScript is **dynamically typed**, meaning variable types are decided at runtime.
- It is **event-driven** and **single-threaded**, using an **event loop** for handling tasks, and to avoid complexities of multi-threaded programming.
- JavaScript works together with **HTML and CSS** to build interactive web applications.



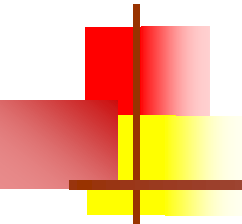
Uses of JavaScript:

- HTML Form validation
- Dynamic content updates (AJAX)
- Interactive web pages
- Frontend frameworks (React, Angular, Vue)
- Backend development (Node.js)
- JavaScript adds **behavior and interactivity** to web pages



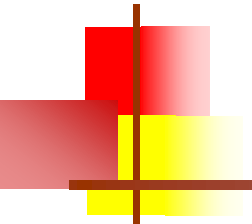
Java Vs JavaScript

Feature	Java	JavaScript
Type	Object-Oriented Programming language	Scripting language (multi-paradigm)
Execution	Compiled → Interpreted → Bytecode → JVM	Interpreted / JIT-compiled by browser or engine
Platform	Platform-independent (JVM)	Browser-dependent / Runtime-dependent
Syntax	Similar to C/C++	Similar to Java but more flexible
Typing	Statically typed	Dynamically typed
Main Use	Backend, desktop, Android apps	Web pages, frontend & backend (Web & Mobile apps)
Memory Management	Automatic Garbage Collection	Automatic Garbage Collection
Concurrency	Multi-threaded	Single-threaded (event loop)
Inheritance	Class-based	Prototype-based
Security	Strongly secure	Less secure (runs in browser)
File Extension	.java	.js



Where to put JavaScript

- JavaScript can be placed/included in **three main locations** in a web page:
 1. Inside the `<script>` tag in the HTML file
 2. In an external JavaScript file
 3. Inline JavaScript (inside HTML elements)
- The primary function of early JavaScript was to manipulate the Document Object Model (DOM) and respond to events (like button clicks) on a single web page. Requiring developers to manage multiple threads accessing and modifying the same UI elements simultaneously would have introduced significant complexity and potential bugs, as synchronization mechanisms would be needed to prevent conflicts.



Inside the `<script>` tag

- In the `<body>` section of HTML page

```
<body>
```

```
<h1>My Page</h1>
```

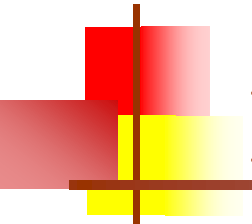
```
<script type="text/javascript" >
```

```
  alert("Page Loaded");
```

```
</script>
```

```
</body>
```

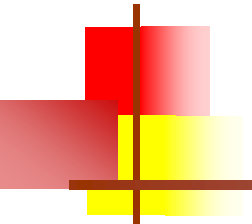
- Executes after page content loads
- Recommended for beginners



External JavaScript file

- JavaScript is written in `<scriptFile>.js`
- Improves readability and reuse
- Easier maintenance
- Best practice for real projects.

`<script src="script.js"></script>`



Inline JavaScript

- Javascript code is inside HTML elements
- Used for small actions
- Not recommended for large programs

`<button onclick="alert('Clicked!')">Click Me</button>`

- Best Practices:
 - Use external JavaScript files
 - Place `<script>` just before `</body>` tag
 - Avoid inline JavaScript when possible



Primitive Data Types

- Primitive types store **single, simple values**.

1. Number

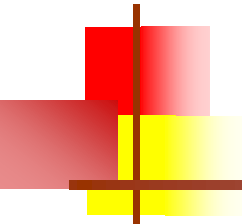
- Stores integers and floating-point numbers
- Also includes **Infinity**, **-Infinity**, and **NaN**
- Ex: `let a=10;`

2. String

- Stores text
- Written inside single (' '), double (" "), or backticks (` `)
- Ex: `let name = "JavaScript";`

3. Boolean (**true** or **false**)

- 0, "0", empty strings, undefined, null, and NaN are **false** , other values are **true**
- Ex: `let isLoggedIn = true;`



Primitive Data Types

4. Undefined

- A variable declared but not assigned a value
 - Ex: `let x;`

5. Null

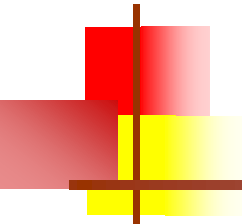
- Represents **intentional absence of value**
 - Ex: `let y = null;`

6. BigInt

- Used for very large integers
 - Ex: `let big = 12345678901234567890;`

7. Symbol

- It is used to create unique and immutable identifiers, mainly as object property keys
 - Ex: `let id = Symbol("userId");`



Non-Primitive (Reference) Data Types

- These store multiple values and are stored by reference.

1. Object

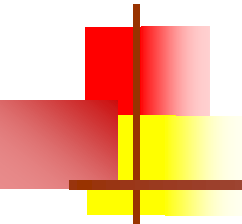
- Collection of key-value pairs

```
let student = {  
  name: "Alex",  
  age: 20  
};
```

2. Array

- Ordered collection of values

```
let colors = ["red", "green", "blue"];
```



Non-Primitive (Reference) Data Types

3. Function

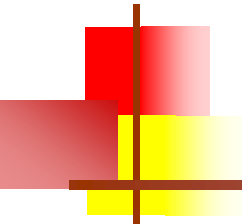
- A block of reusable code

```
function add(a, b) {  
    return a + b;  
}
```

■ typeof Operator

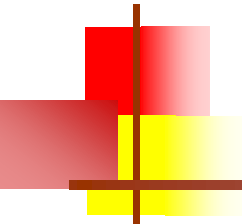
- Used to check the data type of a variable.

```
typeof 10      // "number"  
typeof "Hi"    // "string"  
typeof true    // "boolean"  
typeof null    // "object"
```



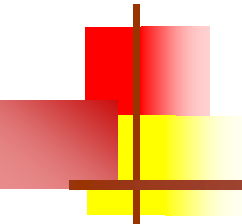
Primitive and Non-Primitive

Feature	Primitive	Non-Primitive
Value stored	Actual value	Reference
Mutable	✗ No	✓ Yes
Examples	number, string	object, array



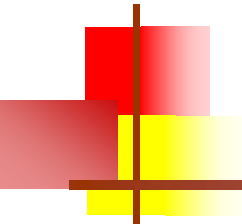
Variables

- Variables are used to store data values that can be used and changed during program execution.
- In JavaScript, variables can be declared using **three keywords**:
 - **var**
 - Old way of declaring variables
 - Function-scoped
 - Can be re-declared and updated
 - Ex: `var x = 10;`
 - **let**
 - Modern and recommended
 - Block-scoped
 - Can be updated but not re-declared in the same scope
 - Ex: `let y = 20;`



Variables (Contd...)

- **const**
 - Used for constant values
 - Block-scoped
 - Cannot be updated or re-declared
 - Ex: `const pi = 3.14;`
- **Rules for Naming Variables:**
 - Must start with a letter, `_`, or `$`
 - Cannot start with a number
 - Cannot use JavaScript reserved keywords
 - Case-sensitive (`age` and `Age` are different)
- Variables are dynamically typed (data type of a variable can be changed during execution)



Variables...

- JavaScript variables can store the following data types:
 - Numbers
 - Strings
 - Boolean values
 - Objects
 - Arrays
 - Functions

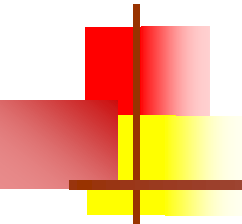
Ex:

```
let name = "Alex";
```

```
let age = 21;
```

```
let isStudent = true;
```

```
Let arr =[1, 2, 4, "sai"];
```



Variables...

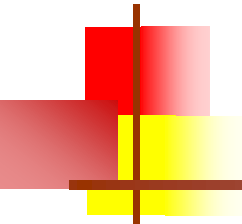
■ Variable Scope

- Global scope – Accessible anywhere (Variables declared outside a function are **global**)
- Local (Block) scope – Accessible only inside { }
- Variables can also be declared implicitly, simply by assigning a value to them
 - Implicitly declared variables are always global

- **Hoisting** (JavaScript's default behavior of moving declarations to the top of their scope (during the compilation phase) before code execution.)
 - **var** variables are hoisted (declared at top)
 - **let** and **const** are hoisted but not initialized

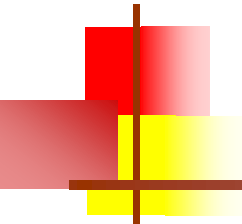
Best Practices:

- Use **let** for variables
- Use **const** for fixed values
- Avoid **var** in modern JavaScript



Operators, I

- Because most JavaScript syntax is borrowed from C (and is therefore just like Java), we'll go through it quickly
- Arithmetic operators (all numbers are floating-point):
+ - * / % ++ --
- Comparison operators:
< <= == != >= >
- Logical operators:
&& || ! (&& and || are *short-circuit* operators)
- Bitwise operators (avoid):
& | ^ ~ << >> >>>
- Assignment operators:
+= -= *= /= % = <<= >>= >>>= &= ^= |=



Operators, II

- String operator:

+

- The conditional operator:

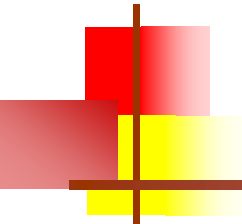
condition ? value_if_true : value_if_false

- Special equality tests:

- == and != try to convert their operands to the same type before performing the test
 - The rules for how this occurs are complex and intransitive; you should avoid using these operators
- === and !== consider their operands *unequal* if they are of different types
 - These operators work like == and != in C or Java

- Additional operators:

new typeof void delete



Equality Tests in JavaScript

- JavaScript provides **three main ways** to compare values for equality.

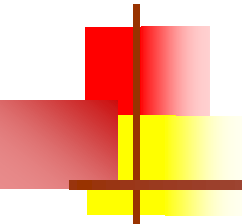
1. `==` (Loose Equality)

- Compares values only
- Performs type conversion (type coercion) before comparison
- Can give **unexpected results** due to automatic type conversion

```
5 == "5"    // true
```

```
true == 1   // true
```

```
null == undefined // true
```



Equality Tests

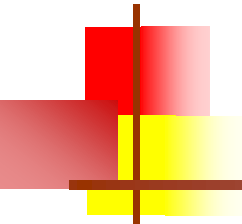
2. === (Strict Equality)

- Compares both value and data type
- No type conversion
- Recommended way
- Predictable and safe
- Best practice in modern JavaScript

```
5 === "5"    // false
```

```
true === 1   // false
```

```
null === undefined // false
```



Equality Tests

3. `Object.is()` (Same-value Equality)

- Compares values more precisely
- Similar to `===`, but handles special cases differently

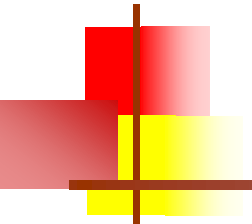
`Object.is(5, 5)` `// true`

`Object.is(NaN, NaN)` `// true`

`Object.is(0, -0)` `// false`

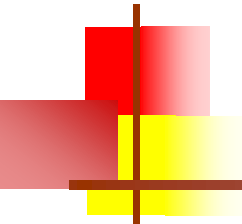
■ Special Comparison Cases:

Comparison	Result	Reason
<code>NaN == NaN</code>	false	NaN is not equal to anything
<code>NaN === NaN</code>	false	Same reason
<code>Object.is(NaN, NaN)</code>	true	Special handling
<code>null == undefined</code>	true	Loose equality rule
<code>null === undefined</code>	false	Different types



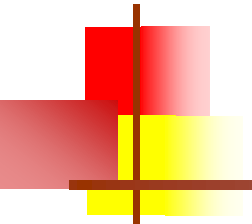
Comments

- Comments are as in C or Java:
 - Between `//` and the end of the line
 - Between `/*` and `*/`



Statements, I

- Most JavaScript statements are also borrowed from C
 - Assignment: `greeting = "Hello, " + name;`
 - Compound statement:
`{ statement; ...; statement }`
 - If statements:
`if (condition) statement;`
`if (condition) statement; else statement;`
 - Familiar loop statements:
`while (condition) statement;`
`do statement while (condition);`
`for (initialization; condition; increment) statement;`



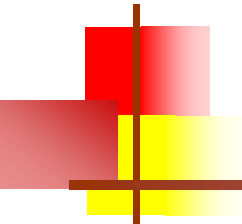
Statements, II

- The switch statement:

```
switch (expression) {  
    case label :  
        statement;  
        break;  
    case label :  
        statement;  
        break;  
    ...  
    default : statement;  
}
```

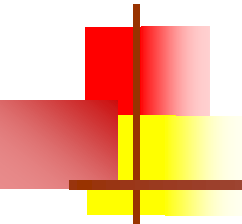
- Other familiar statements:

- break;
- continue;
- The empty statement, as in ;; or { }



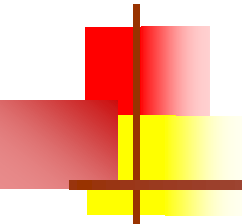
Exception handling, I

- Exception handling in JavaScript is *almost* the same as in Java
- **throw** *expression* creates and throws an exception
 - The *expression* is the value of the exception, and can be of *any* type (often, it's a literal String)
- **try** {
 statements to try
} catch (*e*) { // Notice: no type declaration for *e*
 exception handling statements
} finally { // optional, as usual
 code that is always executed
}
- With this form, there is *only one* **catch** clause



Exception handling, II

- try {
 statements to try
} catch (*e* if *test1*) {
 exception handling for the case that test1 is true
} catch (*e* if *test2*) {
 exception handling for when test1 is false and test2 is true
} catch (*e*) {
 exception handling for when both test1 and test2 are false
} finally { // optional, as usual
 code that is always executed
}
- Typically, the test would be something like
 e == "InvalidNameException"
but it could be any kind of test

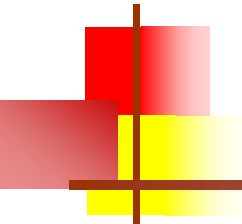


JavaScript Objects

- An object is a collection of related data and functions, stored as key–value pairs.
 - Keys → properties
 - Functions inside objects → methods
 - Objects are non-primitive (reference) data types

Ex:

```
let person = {  
  name: "Alex",  
  age: 22  
};
```

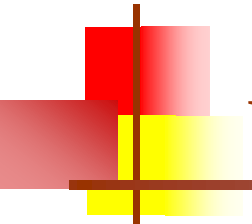


Object Literal

- The **object literal** is the **simplest and most commonly used** way to create an object.

Ex:

```
let student = {  
  name: "Ravi",  
  rollNo: 101,  
  greet: function () {  
    return "Hello";  
  }  
};  
  
student.name    // dot notation  
student["rollNo"] // bracket notation
```



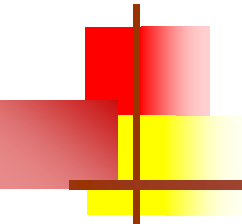
Ways of Creating Objects in JavaScript

1. Using Object Literal (Most Common)

- Simple
- Easy to read
- Best for small objects

Eg:

```
let car = {  
  brand: "Toyota",  
  model: "Innova"  
};
```



Creating Objects in JavaScript

2. Using `new Object()` :

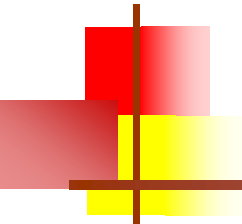
- Less preferred ; Verbose

Ex: `let car = new Object();`
`car.brand = "Toyota";`
`car.model = "Innova";`

3. Using Constructor Function :

- Used when multiple similar objects are needed; Reusable

```
function Student(name, age) {  
    this.name = name;  
    this.age = age;  
}  
  
let s1 = new Student("Anil", 20);  
let s2 = new Student("Sunil", 21);
```



Creating Objects in JavaScript

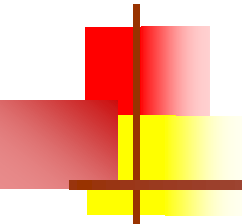
4. Using `Object.create()` :

- Creates a new object using an existing object as a prototype.
- Prototype-based ; Advanced usage

Ex:

```
let person = {  
  greet() {  
    return "Hello";  
  }  
};
```

```
let student = Object.create(person);  
student.name = "Rahul";
```

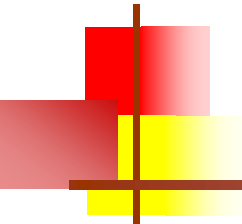


Creating Objects in JavaScript

5. Using ES6 **class** (ECMAScript 6 (ES6), also known as ECMAScript 2015)
- Modern and structured way.
 - Clean syntax; Object-oriented style

Ex:

```
class Student {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
}  
  
let s1 = new Student("Meena", 22);
```

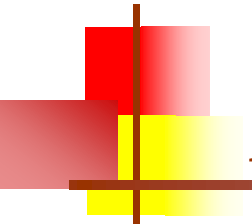


Object Literal vs Constructor

Feature	Object Literal	Constructor
Syntax	Simple	Slightly complex
Reusability	✗ No	✓ Yes
Use case	Single object	Multiple objects

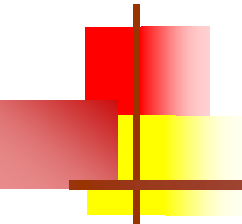
Recap:

- Objects store properties and methods
- Object literal is the easiest way to create objects
- JavaScript supports multiple object-creation techniques
- Objects are passed by reference



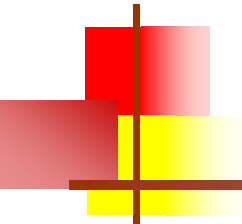
Array literals

- JavaScript has array *literals*, written with brackets and commas
 - Example: `color = ["red", "yellow", "green", "blue"];`
 - Arrays are *zero-based*: `color[0]` is "red"
- If you put two commas in a row, the array has an “empty” element in that location
 - Example: `color = ["red", , , "green", "blue"];`
 - `color` has 5 elements
 - However, a single comma at the end is ignored
 - Example: `color = ["red", , , "green", "blue",];` still has 5 elements



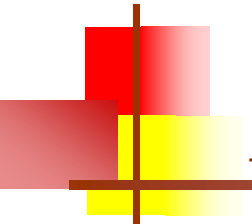
Four ways to create an array

- You can use an array literal:
`var colors = ["red", "green", "blue"];`
- You can use `new Array()` to create an empty array:
 - `var colors = new Array();`
 - You can add elements to the array later:
`colors[0] = "red"; colors[2] = "blue"; colors[1]="green";`
- You can use `new Array(n)` with a single numeric argument to create an array of that size
 - `var colors = new Array(3);`
- You can use `new Array(...)` with *two or more* arguments to create an array containing those values:
 - `var colors = new Array("red", "green", "blue");`



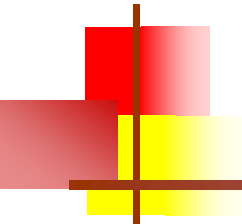
The length of an array

- If `myArray` is an array, its length is given by `myArray.length`
- Array length can be changed by assignment beyond the current length
 - Example: `var myArray = new Array(5); myArray[10] = 3;`
- Arrays are **sparse**, that is, space is only allocated for elements that have been assigned a value
 - Example: `myArray[50000] = 3;` is perfectly OK
 - But indices must be between 0 and $2^{32}-1$
- As in C and Java, there are no two-dimensional arrays; but you can have an array of arrays: `myArray[5][3]`



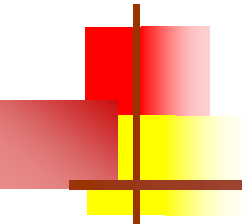
Arrays and objects

- Arrays *are* objects
- `car = { myCar: "Saturn", 7: "Mazda" }`
 - `car[7]` is the same as `car.7`
 - `car.myCar` is the same as `car["myCar"]`
- If you *know* the name of a property, you can use dot notation: `car.myCar`
- If you *don't know* the name of a property, but you have it in a variable (or can compute it), you *must* use array notation: `car["my" + "Car"]`



Array functions

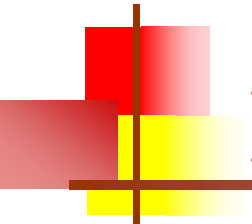
- If `myArray` is an array,
 - `myArray.sort()` sorts the array alphabetically
 - `myArray.sort(function(a, b) { return a - b; })` sorts numerically
 - `myArray.reverse()` reverses the array elements
 - `myArray.push(...)` adds any number of new elements to the end of the array, and increases the array's length
 - `myArray.pop()` removes and returns the last element of the array, and decrements the array's length
 - `myArray.toString()` returns a string containing the values of the array elements, separated by commas (but not enclosed in brackets)



The for...in loop

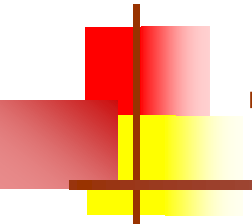
- This is similar to Java's `for(type var : collection)` loop
- You can loop through the properties of an object with `for (variable in object) statement;`
 - Example:

```
for (var prop in course) {  
    document.write(prop + ": " + course[prop]);  
}
```
 - Possible output: `teacher: Dr. Sai`
`number: CS1001`
 - The properties are accessed in an *undefined* order
 - If you add or delete properties of the object within the loop, it is *undefined* whether the loop will visit those properties
 - Arrays *are* objects; applied to an array, `for...in` will visit the “properties” `0, 1, 2, ...`
 - Notice that `course["teacher"]` is equivalent to `course.teacher`
 - You must use brackets if the property name is *in a variable*



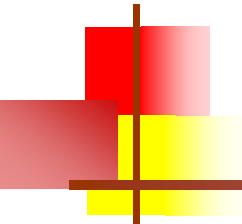
More about the `for...in` loop

- The `for...in` loop does not loop through *all* properties of an object
 - Built-in methods, and many built-in properties, are flagged as *nonenumerable*
 - All built-in properties of functions are nonenumerable
 - There are lots of little surprises like this in JavaScript ☹



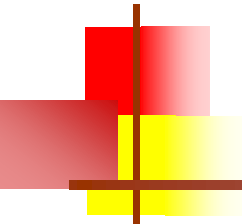
The with statement

- **with** (*object*) *statement* ; uses the *object* as the default prefix for variables in the *statement*
 - If the accessed field doesn't exist, the prefix isn't used
- For example, the following are equivalent:
 - `with (document.myForm) {
 result.value = compute(myInput.value) ;
}`
 - `document.myForm.result.value =
 compute(document.myForm.myInput.value);`
- The **with** statement is useful when you are doing a lot of manipulation on the same object
 - The **with** statement can be confusing, and should be used with care, or avoided entirely



Functions I

- In JavaScript, functions are *values*, and can be assigned, passed as parameters, etc., just like other values
- Within a program, you can use
`var foo = function foo() {...}`
instead of
`function foo() {...}`
- The first form should be preferred for a couple of reasons:
 - The first form makes it clear that `foo` is a value
 - The second form will cause the function to be *hoisted*: moved to the top of the scope in which it is defined
 - The second form cannot be the first thing in a statement
 - The second form is illegal inside an `if` statement, but most browsers allow it anyway (and do different things with it)



Regular expressions

- A regular expression can be written in either of two ways:
 - Within slashes, such as `re = /ab+c/`
 - With a constructor, such as `re = new RegExp("ab+c")`
- Regular expressions are almost the same as in Perl or Java (only a few unusual features are missing)
- `string.match(regex)` searches `string` for an occurrence of `regex`
 - It returns `null` if nothing is found
 - If `regex` has the `g` (global search) flag set, `match` returns an array of matched substrings
 - If `g` is not set, `match` returns an array whose 0th element is the matched text, extra elements are the parenthesized subexpressions, and the `index` property is the start position of the matched substring